Basic MATLAB: Operator

Aritmetika, Relasional, Logika, dan Operator Lain

Andreas Syaloom Kurniawan

Oktober 2026

Fakultas Ekonomi dan Bisnis, Universitas Gadjah Mada

Operator Aritmetika

Apa itu operator aritmetika?

Simbol untuk operasi hitung. Kita mulai dari angka tunggal (skalar), misalnya `a = 6` dan `b = 2`.

Operator	Arti	Contoh	Hasil
+	tambah	`a + b`	8
-	kurang	`a - b`	4
*	kali	`a * b`	12
/	bagi	`a / b`	3
^	pangkat	`a ^ b`	36

Mencoba di Command Window

```
>> a = 6;
>> b = 2;
>> a + b
ans =
     8
>> c = a * b
c =
    12
```

- `=` menyimpan nilai ke variabel (`a` berisi 6).
- `;` di akhir baris: hasil tidak ditampilkan.
- Tanpa `;`: hasil langsung tampil.
- Hasil tanpa nama variabel otomatis disimpan di `ans`.

Pembagian: bagi kanan `/` dan bagi kiri `\`

MATLAB punya dua operator bagi. Bedanya: angka mana yang menjadi pembagi.

Bagi kanan `/` (yang biasa)

```
>> 6 / 2
ans =
     3
```

dibaca: 6 dibagi 2 $(6 \div 2)$

Bagi kiri `\`

```
>> 6 \ 2
ans =
    0.3333
```

dibaca: 2 dibagi 6 $(2 \div 6)$

Cara ingat: ujung atas garis miring selalu condong ke pembagi. `6/2` condong ke 2; `6\2` condong ke 6. Untuk angka tunggal pakai `/` saja; kegunaan `\` baru terasa di matriks.

Dari angka tunggal ke matriks

```
>> A = [1 2; 3 4]
A =
     1     2
     3     4
```

- `[ ]` membuat matriks.
- Spasi (atau koma) memisahkan kolom.
- Titik koma `;` memisahkan baris.

Dalam notasi matematika:

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \qquad B = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$$

`A` berukuran 2×2: 2 baris, 2 kolom.
`B = [5 6; 7 8]` dibuat dengan cara yang sama.

Tambah dan kurang pada matriks

Tambah dan kurang selalu dihitung per posisi (elemen dengan elemen).

```
>> A + B
ans =
     6     8
    10    12

>> A + 10   % skalar ke semua elemen
ans =
    11    12
    13    14
```

$$A + B = \begin{bmatrix} 1+5 & 2+6 \\ 3+7 & 4+8 \end{bmatrix} = \begin{bmatrix} 6 & 8 \\ 10 & 12 \end{bmatrix}$$

Syarat: ukuran `A` dan `B` sama, atau salah satunya angka tunggal.

Perkalian: * (matriks) vs .* (per elemen)

`A * B`: perkalian matriks

baris `A` × kolom `B`, lalu dijumlah

$$\begin{bmatrix} 1\cdot5+2\cdot7 & 1\cdot6+2\cdot8 \\ 3\cdot5+4\cdot7 & 3\cdot6+4\cdot8 \end{bmatrix} = \begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix}$$

Syarat: jumlah kolom `A` = jumlah baris `B`.

`A .* B`: per elemen

posisi yang sama dikalikan

$$\begin{bmatrix} 1\cdot5 & 2\cdot6 \\ 3\cdot7 & 4\cdot8 \end{bmatrix} = \begin{bmatrix} 5 & 12 \\ 21 & 32 \end{bmatrix}$$

Syarat: ukuran sama (atau salah satunya skalar).

Titik (`.`) di depan operator artinya hitung per elemen. Untuk angka tunggal, `*` dan `.*` hasilnya sama.

Pangkat: ^ (matriks) vs .^ (per elemen)

`A ^ 2` sama dengan `A * A`

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} = \begin{bmatrix} 7 & 10 \\ 15 & 22 \end{bmatrix}$$

Hanya untuk matriks persegi.

`A .^ 2` tiap elemen dikuadratkan

$$\begin{bmatrix} 1^2 & 2^2 \\ 3^2 & 4^2 \end{bmatrix} = \begin{bmatrix} 1 & 4 \\ 9 & 16 \end{bmatrix}$$

Ini yang biasanya dimaksud saat mengolah data.

Bagi kanan dan bagi kiri pada matriks

Sistem persamaan $2x + y = 3$ dan $x + 3y = 5$ dapat ditulis $A\mathbf{x} = \mathbf{b}$.

Bagi kiri

`x = A\b` menyelesaikan `A*x = b`

```
>> A = [2 1; 1 3];
>> b = [3; 5];
>> x = A \ b
x =
    0.8000
    1.4000
```

Bagi kanan

`x = b/A` menyelesaikan `x*A = b`
(versi baris dari kasus di kiri)

```
>> [3 5] / A
ans =
    0.8000    1.4000
```

Versi per elemen:
`[2 4 6] ./ 2` → `[1 2 3]`
`[2 4 6] .\ 2` → `[1 0.5 0.3333]`

Pakai `A\b`, bukan `inv(A)*b`: lebih cepat dan lebih akurat.

Transpose ': baris jadi kolom, kolom jadi baris

Vektor 3×1

```
>> v = [1; 2; 3];
>> v'
ans =
     1     2     3
```

ukuran $3 \times 1 \rightarrow 1 \times 3$

Matriks 2×3

```
>> M = [1 2 3; 4 5 6];
>> M'
ans =
     1     4
     2     5
     3     6
```

ukuran $2 \times 3 \rightarrow 3 \times 2$. Cek dengan `size(M')`.

Catatan: ada juga `.'`. Untuk angka riil hasilnya sama dengan `'`; bedanya hanya pada bilangan kompleks.

Urutan prioritas

Dari yang paling dulu dihitung:

1. kurung `( )`
2. transpose dan pangkat `' ^ .^`
3. tanda minus di depan angka
4. `* / .* ./ \`
5. `+ -`
6. titik dua `:`

Setingkat: dihitung dari kiri ke kanan.

```
2 + 3 * 4   % 14, bukan 20
(2 + 3) * 4 % 20

-2^2        % -4  (pangkat dulu)
(-2)^2      %  4

2^3^2       % 64  (kiri ke kanan)
```

Kalau ragu, pakai kurung.

Operator Relasional

Apa itu operator relasional?

Operator untuk membandingkan dua nilai. Hasil operasi dengan operator relasional selalu berupa nilai biner: `1` artinya **benar**, `0` artinya **salah**.

Operator	Dibaca	Contoh (`a = 5`, `b = 3`)	Hasil
`==`	sama dengan	`a == b`	0
`~=`	tidak sama dengan	`a ~= b`	1
`<`	kurang dari	`a < b`	0
`<=`	kurang dari atau sama dengan	`a <= b`	0
`>`	lebih dari	`a > b`	1
`>=`	lebih dari atau sama dengan	`a >= b`	1

Mencoba di Command Window

```
>> a = 5;
>> b = 3;
>> a > b
ans =
  logical
   1
>> t = (a == b)
t =
  logical
   0
```

- 5 lebih dari 3 itu **benar**, jadi hasilnya `1`.
- 5 sama dengan 3 itu **salah**, jadi hasilnya `0`.
- Hasilnya bertipe `logical`: tipe data khusus yang isinya hanya 1 atau 0. Cek dengan `class(t)`.
- Nilai logical tetap bisa dihitung: `(a > b) + 1` hasilnya `2`.

Dua hal yang sering tertukar

1. = bukan ==

```
a = 3      % MENGISI a dengan 3
a == 3     % BERTANYA: apakah a sama
           % dengan 3?  -> 1
```

Satu tanda sama dengan untuk menyimpan, dua untuk membandingkan. Menulis `if a = 3` akan error.

2. Tidak sama dengan: ~=

```
5 ~= 3     % 1  (benar)
5 != 3     % error di MATLAB
```

Di Python atau C ditulis `!=`. Di MATLAB tanda “tidak” adalah ~ (tilde).

Membandingkan vektor: dicek per elemen

Kalau yang dibandingkan vektor, MATLAB mengecek setiap elemen satu per satu. Hasilnya vektor 1/0 dengan ukuran yang sama.

```
>> x = [4 9 1 7];
>> x > 5
ans =
  1x4 logical array
   0   1   0   1
```

Posisi	1	2	3	4
x	4	9	1	7
lebih dari 5?	tidak	ya	tidak	ya
`x > 5`	0	**1**	0	**1**

Dua vektor juga bisa dibandingkan asal ukurannya sama: `[4 9 1 7] == [4 0 1 0]` → `[1 0 1 0]`.

Ingat dulu: mengambil elemen berdasarkan posisi

Selama ini kita mengambil elemen dengan menyebut nomor posisinya.

```
>> x = [4 9 1 7];
>> x(2)
ans =
     9
>> x([2 4])
ans =
     9     7
```

Posisi	1	2	3	4
x	4	**9**	1	**7**

`x([2 4])` artinya: ambil elemen di posisi 2 dan posisi 4.

Masalahnya: kalau datanya ribuan, kita tidak tahu posisi mana yang nilainya lebih dari 5.

Logical indexing: mengambil elemen dengan vektor 1/0

Selain dengan nomor posisi, elemen bisa diambil dengan vektor logical yang panjangnya sama: elemen di posisi bernilai `1` diambil, posisi bernilai `0` dilewati.

Posisi	1	2	3	4
`x`	4	9	1	7
`idx`	0	1	0	1
diambil?	–	**9**	–	**7**

```
>> idx = x > 5;
>> x(idx)
ans =
     9     7
```

Inilah yang disebut logical indexing: memilih data berdasarkan syarat, bukan berdasarkan posisi.

Logical indexing dalam satu baris

Dua langkah tadi biasanya ditulis sekaligus:

```
% dua langkah
idx = x > 5;
x(idx)

% satu langkah (lebih umum)
x(x > 5)
```

Cara membaca `x(x > 5)`:

1. hitung dulu yang di dalam kurung: `x > 5` → `[0 1 0 1]`
2. lalu ambil elemen `x` di posisi yang bernilai 1 → `[9 7]`

Dibaca: “elemen `x` yang lebih dari 5”.

Catatan: vektornya harus bertipe logical. `x([0 1 0 1])` dengan angka biasa akan error, karena posisi 0 tidak ada.

Kegunaan sehari-hari

```
x = [4 9 1 7];

x(x > 5)        % ambil nilainya
% [9 7]

find(x > 5)     % di posisi mana?
% [2 4]

x(x > 5) = 0    % ganti nilainya
% [4 0 1 0]
```

```
x = [4 9 1 7];

sum(x > 5)      % ada berapa?
% 2

mean(x > 5)     % berapa proporsinya?
% 0.5000
```

`sum` dan `mean` bekerja karena 1/0 bisa dijumlahkan: jumlah angka 1 = banyaknya elemen yang memenuhi syarat.

Jebakan: angka desimal dan teks

Angka desimal

```
>> 0.1 + 0.2 == 0.3
ans =
  logical
   0
```

Komputer menyimpan desimal dengan sedikit pembulatan, jadi `0.1 + 0.2` tidak persis 0.3. Bandingkan dengan toleransi:

`abs((0.1+0.2) - 0.3) < 1e-10` $\rightarrow$ 1

Teks

```
>> 'abc' == 'abd'
ans =
  1x3 logical array
   1   1   0
```

== pada teks membandingkan huruf per huruf, dan error kalau panjangnya beda. Untuk teks pakai `strcmp('abc','abd')` $\rightarrow$ 0.

Membandingkan dua matriks secara utuh: `isequal(A, B)`.

Operator Logika

Apa itu operator logika?

Operator relasional menghasilkan satu syarat (1 atau 0). Operator logika dipakai untuk menggabungkan beberapa syarat atau membalik sebuah syarat. Masukan dan hasilnya sama-sama 1 (benar) atau 0 (salah).

Operator	Dibaca	Benar jika ...
`&`	dan	kedua syarat benar
`|`	atau	minimal satu syarat benar
`~`	tidak	syaratnya salah (dibalik)
`&&`	dan (short-circuit)	seperti `&`, khusus satu nilai, untuk `if`/`while`
`||`	atau (short-circuit)	seperti `|`, khusus satu nilai, untuk `if`/`while`
`xor`	salah satu saja	tepat satu syarat benar, tidak boleh dua-duanya

Kenapa perlu? Contoh: “apakah x di antara 2 dan 8?”

Cara matematika: salah di MATLAB

```
>> x = 10;
>> 2 < x < 8
ans =
  logical
   1          % padahal 10 > 8 !
```

MATLAB menghitung dari kiri: `2 < 10` $\rightarrow$ 1, lalu `1 < 8` $\rightarrow$ 1.

Cara yang benar: dua syarat digabung

```
>> (x > 2) & (x < 8)
ans =
  logical
   0
```

Syarat 1: `x > 2` $\rightarrow$ 1
Syarat 2: `x < 8` $\rightarrow$ 0
Digabung dengan “dan” $\rightarrow$ 0. Benar.

`&` (dan): benar jika *kedua* syarat benar

Contoh: mahasiswa lulus jika nilai teori ≥ 60 **dan** nilai praktik ≥ 60.

A	B	A & B
0	0	0
0	1	0
1	0	0
1	1	**1**

```
teori   = [70 50 80];
praktik = [65 90 55];

lulus = (teori >= 60) & ...
        (praktik >= 60)
% lulus = [1 0 0]
```

Mhs	1	2	3
teori ≥ 60	1	0	1
praktik ≥ 60	1	1	0
&	**1**	0	0

Satu nilai: `(5 > 3) & (2 > 4)` $\rightarrow$ `1 & 0` $\rightarrow$ `0`. Bentuk fungsi: `and(A, B)`.

`|` (atau): benar jika *minimal satu* syarat benar

Contoh: mahasiswa remedial jika nilai teori < 60 **atau** nilai praktik < 60.

A	B	A | B
0	0	0
0	1	**1**
1	0	**1**
1	1	**1**

```
teori   = [70 50 80];
praktik = [65 90 55];

rem = (teori < 60) | ...
      (praktik < 60)
% rem = [0 1 1]
```

Mhs	1	2	3
teori < 60	0	1	0
praktik < 60	0	0	1
|	0	**1**	**1**

Satu nilai: `(5 > 3) | (2 > 4)` $\rightarrow$ `1 | 0` $\rightarrow$ `1`. Bentuk fungsi: `or(A, B)`.

~ (tidak): membalik syarat

Benar jadi salah, salah jadi benar. Contoh: siapa yang tidak lulus?

A	~A
0	**1**
1	0

```
lulus = [1 0 0];
~lulus
% [0 1 1]

~(5 > 3)    % ~1
% 0
```

Hasil `~lulus` sama dengan `rem` di slide sebelumnya: “tidak lulus” = “remedial”.

Tanda ~ yang sama dipakai di ~= (tidak sama dengan). Bentuk fungsi: `not(A)`.
Angka selain 0 dianggap benar: `~5` → 0, `~0` → 1.

`&&` (dan, short-circuit): idenya

Analogi: satpam gedung mengecek dulu apakah tamu membawa KTP. Kalau tidak bawa, tamu langsung ditolak; satpam tidak perlu mencari nama di daftar tamu.

Aturannya

- Syarat kiri dicek lebih dulu.
- Kiri **salah** $\rightarrow$ hasil pasti 0, syarat kanan dilewati.
- Kiri **benar** $\rightarrow$ syarat kanan baru dicek.

Bedanya dengan `&`

- `&` selalu mengecek dua-duanya.
- `&&` hanya untuk satu nilai di tiap sisi, jadi dipakai di `if` dan `while`.
- Kegunaan utamanya: syarat kiri melindungi syarat kanan dari error.

&&: contoh mengecek nilai mahasiswa ke-k

Data nilai hanya ada untuk 3 mahasiswa. Kita ingin mengecek apakah mahasiswa ke-k lulus.

```
nilai = [70 50 80];   % 3 mahasiswa
k = 5;                % mahasiswa ke-5 tidak ada

if k <= numel(nilai) && nilai(k) >= 60
    disp('Lulus')
else
    disp('Tidak lulus / tidak ada')
end
% Tidak lulus / tidak ada
```

Kalau && diganti &: error, *Index exceeds the number of array elements*.

	k = 2	k = 5
k <= 3	1	0
nilai(k) >= 60	0	*dilewati*
hasil &&	0	0

numel(nilai) = banyaknya elemen (3).

Saat k = 5, syarat kiri sudah salah, jadi nilai(5) tidak pernah dijalankan.

|| (atau, short-circuit): idenya

Analogi: mahasiswa boleh ikut ujian jika kehadirannya $\geq 75\%$ **atau** punya surat izin. Kalau kehadirannya sudah cukup, dosen tidak perlu memeriksa surat izin.

Aturannya

- Syarat kiri dicek lebih dulu.
- Kiri **benar** $\rightarrow$ hasil pasti 1, syarat kanan dilewati.
- Kiri **salah** $\rightarrow$ syarat kanan baru dicek.

```
hadir = 80;      % persen
izin  = false;

if hadir >= 75 || izin
    disp('Boleh ikut ujian')
end
% hadir >= 75 -> 1,
% izin tidak dicek lagi
```

`||`: contoh mencari mahasiswa yang perlu ditindaklanjuti

Mahasiswa ke-`k` perlu ditindaklanjuti jika datanya tidak ada **atau** nilainya di bawah 60.

```
nilai = [70 50 80];
k = 5;

if k > numel(nilai) || nilai(k) < 60
    disp('Perlu ditindaklanjuti')
else
    disp('Aman')
end
% Perlu ditindaklanjuti
```

	`k = 1`	`k = 2`	`k = 5`
`k > 3`	0	0	1
`nilai(k) < 60`	0	1	*dilewati*
hasil `||`	0	1	1

Saat `k = 5`, syarat kiri sudah benar, jadi `nilai(5)` tidak dijalankan. Dengan `|` biasa, baris ini error.

`xor` (salah satu saja)

Benar jika tepat satu syarat benar; kalau dua-duanya benar justru salah. Berupa fungsi, tidak ada simbolnya. Contoh: siapa yang lulus hanya satu ujian?

A	B	xor
0	0	0
0	1	**1**
1	0	**1**
1	1	0

```
teori   = [70 50 80];
praktik = [65 90 55];

satu = xor(teori >= 60, ...
           praktik >= 60)
% satu = [0 1 1]
```

Mhs	1	2	3
teori ≥ 60	1	0	1
praktik ≥ 60	1	1	0
`xor`	0	**1**	**1**

Bandingkan dengan `|`: mahasiswa 1 lulus dua-duanya, jadi `|` $\rightarrow$ 1 tetapi `xor` $\rightarrow$ 0.

Ringkasan: kapan pakai yang mana?

Situasi	Pakai	Contoh
Menyaring data (vektor/matriks)	`& | ~`	`x((x > 2) & (x < 8))`
Syarat di `if` / `while`	`&& || ~`	`if n > 0 && rata > 10`
Tepat satu syarat yang benar	`xor`	`xor(A, B)`

Aturan praktis: kalau datanya lebih dari satu nilai, pakai `&` dan `|` (satu garis). Kalau syaratnya satu nilai untuk mengambil keputusan, pakai `&&` dan `||` (dua garis).

Operator Lain-lain

Operator Lain-lain

Ringkasan

Operator	Fungsi
`:`	range (`1:5`, `0:0.1:1`) atau “semua” dalam indeks (`A(:,2)`)
`( )`	indeks array, argumen fungsi
`{ }`	isi cell array atau isi table (`T{1,1}`)
`[ ]`	membentuk matriks, menggabungkan (`[A B]` horizontal, `[A; B]` vertikal)
`.`	akses field struct atau kolom table (`S.x`, `T.NamaKolom`)
`;`	pisah baris dalam `[ ]`, atau menekan output di akhir baris
`,`	pisah kolom atau argumen
`%`	komentar (`%%` untuk section)
`...`	lanjut ke baris berikutnya

Operator Lain-lain

Titik dua :

Membuat deret

awal:langkah:akhir

```
1:5          % langkah 1 (default)
% [1 2 3 4 5]

0:0.25:1     % langkah 0.25
% [0 0.25 0.5 0.75 1]

10:-2:2      % menurun
% [10 8 6 4 2]
```

Artinya “semua” dalam indeks

```
A = [1 2 3; 4 5 6];

A(:, 2)      % semua baris, kolom 2
% [2; 5]

A(1, :)      % baris 1, semua kolom
% [1 2 3]

A(:)         % semua elemen, 1 kolom
% [1; 4; 2; 5; 3; 6]
```

Operator Lain-lain

Kurung biasa ( )

Untuk mengindeks

```
x = [4 9 1 7];
x(3)
% 1

A = [1 2 3; 4 5 6];
A(2, 1)    % baris 2,
% 4        % kolom 1
```

Untuk argumen fungsi

```
max(x)
% 9

round(3.14159, 2)
% 3.14

zeros(2, 3)
% matriks 0 ukuran 2x3
```

Untuk urutan hitung

```
2 + 3 * 4
% 14

(2 + 3) * 4
% 20
```

Indeks di MATLAB mulai dari 1, bukan 0. Urutannya selalu (baris, kolom).

Kurung kurawal { }

Cell array: wadah yang tiap selnya boleh berisi tipe data berbeda.

Cell array

```
C = {'Yogyakarta', 2023, [1 2 3]};

C{1}      % ISI sel ke-1
% 'Yogyakarta'

C(1)      % SEL ke-1 (masih cell)
% {'Yogyakarta'}
```

Table

```
T = table([1;2], [3;4], ...
    'VariableNames', {'x','y'});

T{1, 2}   % isi baris 1, kolom 2
% 3

T{:, :}   % seluruh isi jadi matriks
% [1 3; 2 4]
```

Cara ingat: `{ }` mengambil isi, `( )` mengambil wadahnya.

Operator Lain-lain

Kurung siku [ ]

Membuat vektor/matriks

```
[1 2 3]      % baris
[1; 2; 3]    % kolom
[1 2; 3 4]   % 2x2
```

Menggabungkan matriks

```
a = [1 2];
b = [3 4];

[a b]     % ke samping
% [1 2 3 4]

[a; b]    % ke bawah
% [1 2; 3 4]
```

Menghapus elemen

```
x = [4 9 1 7];
x(2) = []
% [4 1 7]

y = [];   % kosong
```

Syarat: ke samping jumlah baris sama; ke bawah jumlah kolom sama.

Operator Lain-lain

Titik `.`

Mengakses bagian bernama di dalam struct atau table.

Struct

```
S.nama = 'Budi';
S.umur = 25;

S.nama
% 'Budi'

S.umur + 1
% 26
```

Kolom table

```
T = table([1;2], [3;4], ...
    'VariableNames', {'x','y'});

T.y          % kolom bernama y
% [3; 4]
```

Jangan tertukar dengan `.*` `./` `.^`: di sana titik artinya “per elemen”.

Operator Lain-lain

Titik koma ;

Memisahkan baris matriks

```
[1 2; 3 4]
% ans =
%    1   2
%    3   4
```

Menyembunyikan hasil

```
>> a = 5
a =
     5
>> b = 7;
>>              % tidak tampil
```

Beberapa perintah satu baris

```
a = 1; b = 2;
c = a + b;
```

Wajib di akhir baris? Tidak. Pindah baris sudah mengakhiri perintah. Fungsinya hanya menyembunyikan hasil.

Titik koma ; di dalam function

Secara aturan tidak wajib, tetapi di function hampir selalu dipakai. Tanpa ;, hasil antara ikut tercetak setiap kali function dipanggil.

File `kuadrat_plus.m`

```
function y = kuadrat_plus(x)
z = x^2        % lupa titik koma
y = z + 1;
end
```

Saat dipanggil

```
>> a = kuadrat_plus(3);
z =
     9
```

`z` tetap muncul walau pemanggilnya sudah pakai ;, karena ; di luar tidak berlaku untuk baris di dalam function.

Kebiasaan baik: akhiri setiap baris perhitungan di function dan skrip dengan ;. Hapus ; sementara hanya saat ingin melihat hasil antara (debugging).

Operator Lain-lain

Koma `,`

Memisahkan kolom matriks

```
[1, 2, 3]
% sama dengan [1 2 3]
```

Memisahkan argumen

```
zeros(2, 3)
round(3.14159, 2)
A(2, 3)
% baris 2, kolom 3
```

Memisahkan perintah

```
>> a = 1, b = 2
a =
     1
b =
     2
```

Bedanya dengan `;`: koma memisahkan tanpa menyembunyikan output.

Operator Lain-lain

Komentar `%` dan section `%%`

```
%% Bagian 1: Data
x = [4 9 1 7];   % data contoh

%% Bagian 2: Analisis
rata = mean(x);  % rata-rata

%{
Komentar beberapa baris
sekaligus
%}
```

- Teks setelah `%` diabaikan MATLAB: untuk catatan.
- `%%` di awal baris membagi skrip menjadi section. Satu section bisa dijalankan sendiri dengan **Run Section** (Ctrl+Enter, di Mac Cmd+Enter).
- `%{ ... %}` untuk komentar beberapa baris.

Tiga titik ...

Menyambung perintah yang terlalu panjang ke baris berikutnya.

Rumus panjang

```
total = harga_1 + harga_2 + ...
        harga_3 + harga_4;
```

Matriks panjang

```
x = [1 2 3 4 5 ...
     6 7 8 9 10];
```

Argumen fungsi

```
T = table([1;2], [3;4], ...
    'VariableNames', {'x','y'});
```

MATLAB membaca semuanya sebagai satu perintah. Teks setelah `...` di baris itu dianggap komentar.